

WEEK-4 TASK

High-Performance Market Basket Analytics with Compressed FP-Trees from Scratch

Project Deliverable: Frequent Pattern Mining Engine + High-Confidence Association Rules

1. Objective

Build a frequent-pattern mining engine from scratch using a compressed FP-Tree. The implementation dynamically creates tree nodes, maintains node-link pointers, constructs conditional FP-Trees recursively, mines frequent itemsets, and generates high-confidence association rules. The design avoids external FP-growth libraries.

2. Core Approach

- **Transaction preprocessing:** remove duplicate items in a transaction and count item frequencies.
- **Frequency filtering:** discard items below the minimum support threshold.
- **Global ordering:** order surviving items by descending frequency.
- **Compressed FP-Tree:** insert ordered transactions so common prefixes share nodes.
- **Header table:** store each frequent item and its first node; node-link pointers connect all nodes for that item.
- **Conditional pattern base:** collect prefix paths ending at a selected item.
- **Conditional FP-Tree:** rebuild a smaller tree from the conditional pattern base.
- **Recursive mining:** repeatedly mine conditional trees until no further patterns remain.
- **Association rules:** generate rules from frequent itemsets and calculate confidence.

3. Python Implementation — From Scratch

```
from collections import Counter, defaultdict
from itertools import combinations

class FPNode:
    def __init__(self, item, count, parent):
        self.item = item
        self.count = count
        self.parent = parent
        self.children = {}
        self.link = None

def build_tree(transactions, min_count):
    freq = Counter()
    for t in transactions:
        freq.update(set(t))

    freq = {i:c for i,c in freq.items() if c >= min_count}
    if not freq:
        return None, {}

    root = FPNode(None, 0, None)
    header = {i: [freq[i], None] for i in freq}

    for t in transactions:
        items = [i for i in set(t) if i in freq]
        items.sort(key=lambda i: (-freq[i], str(i)))
        cur = root

        for item in items:
            if item in cur.children:
                child = cur.children[item]
                child.count += 1
            else:
                child = FPNode(item, 1, cur)
                cur.children[item] = child
```

```

        if header[item][1] is None:
            header[item][1] = child
        else:
            node = header[item][1]
            while node.link is not None:
                node = node.link
            node.link = child
        cur = child

    return root, header

def conditional_pattern_base(header_node):
    paths = []
    node = header_node

    while node:
        path = []
        parent = node.parent
        while parent and parent.item is not None:
            path.append(parent.item)
            parent = parent.parent
        if path:
            paths.append((path[::-1], node.count))
        node = node.link

    return paths

def expand_weighted_paths(paths):
    expanded = []
    for path, count in paths:
        for _ in range(count):
            expanded.append(path)
    return expanded

def mine_fp_tree(transactions, min_count, suffix=()):
    root, header = build_tree(transactions, min_count)
    if root is None:
        return []

    patterns = []
    items = sorted(header, key=lambda i: header[i][0])

    for item in items:
        support = header[item][0]
        new_pattern = tuple(sorted(suffix + (item,), key=str))
        patterns.append((new_pattern, support))

        base = conditional_pattern_base(header[item][1])
        conditional_transactions = expand_weighted_paths(base)

        if conditional_transactions:
            patterns.extend(
                mine_fp_tree(conditional_transactions, min_count, suffix + (item,))
            )

    # remove duplicates caused by recursive paths
    unique = {}
    for pattern, support in patterns:
        unique[pattern] = max(unique.get(pattern, 0), support)
    return list(unique.items())

def association_rules(patterns, min_confidence):
    support = dict(patterns)
    rules = []

    for itemset, sup in support.items():
        if len(itemset) < 2:
            continue

        items = list(itemset)
        for r in range(1, len(items)):
            for left in combinations(items, r):
                left = tuple(sorted(left, key=str))
                right = tuple(sorted(set(items) - set(left), key=str))
                conf = sup / support.get(left, 1)
                if conf >= min_confidence:
                    rules.append((left, right, sup, conf))
    return sorted(rules, key=lambda x: (-x[3], -x[2]))

# Example
transactions = [
    ["milk", "bread", "butter"],
    ["bread", "butter"],
    ["milk", "bread"],

```

```

["milk", "eggs"],
["bread", "eggs", "butter"],
["milk", "bread", "eggs"],
]

min_support_count = 2
min_confidence = 0.70

patterns = mine_fp_tree(transactions, min_support_count)
rules = association_rules(patterns, min_confidence)

print("Frequent Itemsets:")
for p, s in sorted(patterns, key=lambda x: (-x[1], x[0])):
    print(p, "support =", s)

print("\nAssociation Rules:")
for left, right, sup, conf in rules:
    print(left, "->", right,
          "support =", sup,
          "confidence =", round(conf, 3))

```

4. Example Output Interpretation

For the sample transactions, the engine first counts item frequency, removes infrequent items, orders each transaction using the global frequency order, and inserts the result into the FP-Tree. It then follows header-table node links to create conditional pattern bases. Rules are retained when their confidence is at least 70%. The exact output can change if the minimum-support or confidence thresholds are changed.

5. Complexity and Performance

FP-Growth is generally efficient because it avoids generating a large candidate set like Apriori. The compressed tree shares common prefixes, reducing repeated transaction storage. The implementation uses hash maps for child lookup and a header table for item-node traversal. For truly massive datasets, the sample implementation can be optimized further by storing transaction counts directly instead of expanding weighted conditional paths, and by maintaining tail pointers for $O(1)$ node-link insertion.

6. Performance Enhancements for the Task

- Use integer item IDs instead of long strings during large-scale mining.
- Use a header-table tail pointer so appending a node to a node-link chain does not require traversal.
- Keep conditional paths weighted rather than physically duplicating them.
- Use iterative preprocessing and streaming input for very large files.
- Profile insertion, conditional-tree construction, and rule generation separately.
- Set support and confidence thresholds according to dataset size to control pattern explosion.

7. Conclusion

This Week-4 solution implements the main FP-Growth pipeline from scratch: frequency counting, compressed FP-Tree construction, header-table node links, conditional pattern-base extraction, recursive conditional FP-Tree mining, frequent-itemset discovery, and confidence-based association rule generation. It provides a clear foundation for benchmarking the engine on larger transaction datasets.

Submission note: Replace the sample transactions with your assigned massive dataset and report runtime, minimum support, minimum confidence, number of frequent itemsets, and number of generated rules.